

PZ-OV2640 摄像头模块开发手册

本手册将教大家如何在玄武&朱雀开发板上使用 PZ-OV2640 摄像头模块。本章分为如下几部分内容：

- 1 PZ-OV2640 模块简介
- 2 硬件设计
- 3 软件设计
- 4 实验现象

1 PZ-OV2640 模块简介

1.1 特性参数

PZ-OV2640 是深圳普中科技推出的一款 200W 像素高清摄像头模块。该模块采用了一颗 OV2640 集成化模组,从而使模块更加精致小巧,而且该模组还可重复利用。PZ-OV2640 模块采用该 OV2640 模组作为核心部件,集成有源晶振和 LDO,接口简单,使用非常方便。

PZ-OV2640 模块的特点如下:

- 高灵敏度、低电压适合嵌入式应用
- 标准的 SCCB 接口,兼容 IIC 接口
- 支持 RawRGB、RGB(RGB565/RGB555)、GRB422、YUV(422/420)和 YCbCr(422)

输出格式

- 支持 UXGA、SXGA、SVGA 以及按比例缩小到从 SXGA 到 40*30 的任何尺寸
- 支持自动曝光控制、自动增益控制、自动白平衡、自动消除灯光条纹、自动黑电平校准等自动控制功能。同时支持色饱和度、色相、伽马、锐度等设置。
- 支持图像缩放、平移和窗口设置
- 支持图像压缩,即可输出 JPEG 图像数据
- 自带嵌入式微处理器
- 集成有源晶振,无需外部提供时钟
- 集成 LDO,仅需提供 3.3V 电源即可正常工作

PZ-OV2640 模块各项参数如下图所示。

项目	说明
接口类型	数据接口：8 位数据 控制接口：SCCB（类 IIC 协议）
输出格式	RawRGB、RGB(RGB565/RGB555)、GRB422、YUV(422/420)、YCbCr(422)、JPEG 数据
输出位宽	8 位
输出像素	UXGA(1600*1200)及以下到 40*30 的任意尺寸
最大帧率	UXGA(1600*1200)@15 帧 SVGA(800*600)@30 帧 CIF(352*288)@60 帧
灵敏度	0.6V/Lux-sec
信噪比	40dB
动态范围	50dB
镜头光圈	F2.0
镜头视角	62°
镜头焦距	可调
工作温度	-30℃~70℃
模块尺寸	24mm*32.98mm
电源电压	3.3V
IO 口电平 ^①	2.8V LVTTTL，可兼容 3.3V
功耗	40mA

注①：模块 IO 电压是 2.8V，不过对于 3.3V 系统，是可以直接兼容的。所以 3.3V 的 MCU 无需任何处理，直接连接模块即可。不过如果是 5V 的 MCU，建议在信号线上串接 1K 左右电阻，做限流处理。

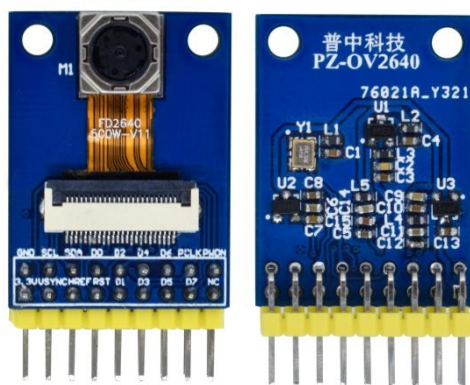
UXGA，即分辨率为 1600*1200 的输出格式，类似的还有：SXGA(1280*1024)、WXGA+(1440*900)、XVGA(1280*960)、WXGA(1280*800)、XGA(1024*768)、SVGA(800*600)、VGA(640*480)、CIF(352*288)、WQVGA(400*240)、QCIF(176*144)和 QQVGA(160*120)等。

1.2 使用说明

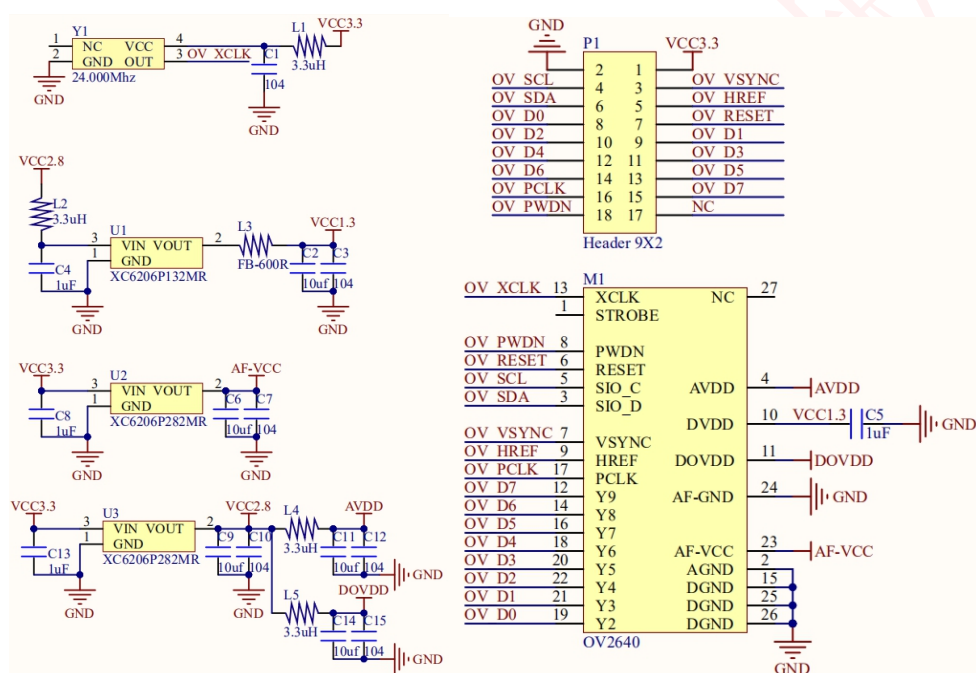
1.2.1 模块引脚说明

PZ-0V2640 摄像头模块通过 2*9 的弯排针（2.54mm 间距）同外部连接，模块可以与玄武&朱雀等开发板直接对接，而普中 F103 最小系统开发板则可以通过杜邦线连接模块进行测试。所有普中 STM32F103 系列开发板都可使用该例程，用户可以直接在这些开发板上，对模块进行测试。

PZ-0V2640 摄像头模块外观如下图所示：



PZ-OV2640 摄像头模块原理图如下图所示：



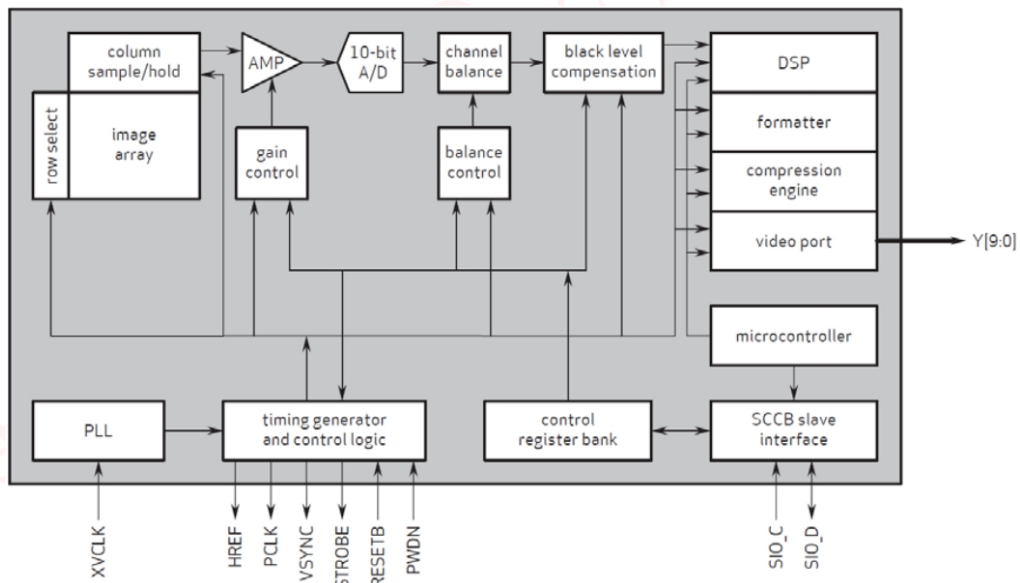
从图中可以看出，模块自带了 1.3V 和 2.8V 的稳压芯片，给 OV2640 供电，因此外部仅需提供 3.3V 电压即可；同时自带了一个 24M 的有源晶振，所以模块不需要外部提供时钟。PZ-OV2640 摄像头模块通过一个 2*9 的排针（P1）同外部电路连接，各引脚的详细描述如下图所示：

序号	名称	说明
1	VCC	3.3V 电源输入脚
2	GND	地线
3	OV_VSYNC	帧同步信号 (OUT ^②)
4	OV_SCL	SCCB 时钟线 (IN ^①)
5	OV_HREF	行参考信号 (OUT)
6	OV_SDA	SCCB 数据线 (IN/OUT)
7	OV_RESET	复位信号 (低电平有效) (IN)
8~15	OV_D0~D7	数据线 (OUT)
16	OV_PCLK	像素时钟 (OUT)
17	NC	未用到
18	OV_PWDN	掉电模式使能 (高电平有效) (IN)

注①、②：IN，表示输入信号；OUT 表示输出信号；IN/OUT 表示即可以输出，也可以做输入信号。从上图可以看出，PZ-OV2640 摄像头模块，总共需要 15 个 IO 口驱动。

1.2.2 OV2640 内部功能结构介绍

OV2640 的功能框图如下图所示：



OV2640 传感器包括如下一些功能模块。

(1) 感光阵列 (Image Array)

OV2640 总共有 1632*1232 个像素，最大输出尺寸为 UXGA (1600*1200)，即 200W 像素。

(2) 模拟信号处理 (Analog Processing)

模拟信号处理所有模拟功能，并包括：模拟放大 (AMP)、增益控制、通道

平衡和平衡控制等。

(3) 10 位 A/D 转换 (A/D)

原始的信号经过模拟放大后，分 G 和 BR 两路进入一个 10 位的 A/D 转换器，A/D 转换器工作频率高达 20M，与像素频率完全同步（转换的频率和帧率有关）。除 A/D 转换器外，该模块还有黑电平校正（BLC）功能。

(4) 数字信号处理器 (DSP)

这个部分控制由原始信号插值到 RGB 信号的过程，并控制一些图像质量：

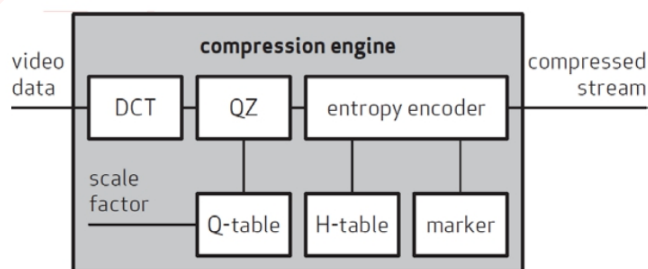
- 边缘锐化（二维高通滤波器）
- 颜色空间转换（原始信号到 RGB 或者 YUV/YCbYCr）
- RGB 色彩矩阵以消除串扰
- 色相和饱和度的控制
- 黑/白点补偿
- 降噪
- 镜头补偿
- 可编程的伽玛
- 十位到八位数据转换

(5) 输出格式模块 (Output Formatter)

该模块按设定优先级控制图像的所有输出数据及其格式。

(6) 压缩引擎 (Compression Engine)

压缩引擎框图如下图所示：



从上图可以看出，压缩引擎主要包括三部分：DCT、QZ 和 entropy encoder（熵编码器），将原始的数据流，压缩成 jpeg 数据输出。

(7) 微处理器 (Microcontroller)

OV2640 自带了一个 8 位微处理器，该处理器有 512 字节 SRAM，4KB 的 ROM，它提供一个灵活的主机到控制系统的指令接口，同时也具有细调图像质量

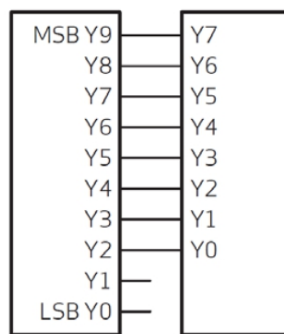
的功能。

(8) SCCB 接口 (SCCB Interface)

SCCB 接口控制图像传感器芯片的运行，详细使用方法参照模块参考资料的《OmniVision Technologies Serial Camera Control Bus (SCCB) Specification》这个文档。

(9) 数字视频接口 (Digital Video Port)

OV2640 拥有一个 10 位数字视频接口(支持 8 位接法)，其 MSB 和 LSB 可以程序设置先后顺序，PZ-OV2640 模块采用默认的 8 位连接方式，如下图所示：



1.2.2 OV2640 传感器窗口介绍

接下来，我们介绍一下 OV2640 的传感器窗口设置、图像尺寸设置、图像窗口设置和图像输出大小设置，这几个设置与我们的正常使用密切相关，有必要了解一下。其中，除了传感器窗口设置是直接针对传感器阵列的设置，其他都是 DSP 部分的设置了，接下来我们一个个介绍。

传感器窗口设置，该功能允许用户设置整个传感器区域（1632*1220）的兴趣部分，也就是在传感器里面开窗，开窗范围从 2*2~1632*1220 都可以设置，不过要求这个窗口必须大于等于随后设置的图像尺寸。传感器窗口设置，通过：0X03/0X19/0X1A/0X07/0X17/0X18 等寄存器设置，寄存器定义请看模块参考资料 OV2640_DS(1.6).pdf 这个文档（下同）。

图像尺寸设置，也就是 DSP 输出（最终输出到 LCD 的）图像的最大尺寸，该尺寸要小于等于前面我们传感器窗口设置所设定的窗口尺寸。图像尺寸通过：0XC0/0XC1/0X8C 等寄存器设置。

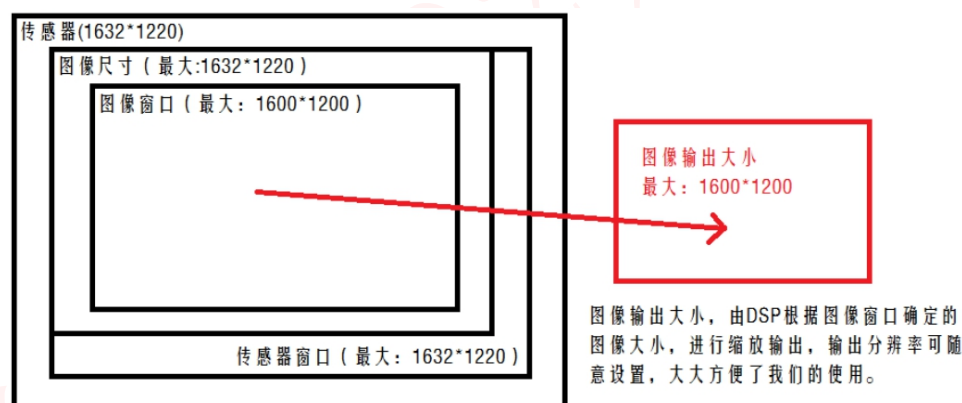
图像窗口设置，这里起始和前面的传感器窗口设置类似，只是这个窗口是在

我们前面设置的图像尺寸里面，再一次设置窗口大小，该窗口必须小于等于前面设置的图像尺寸。该窗口设置后的图像范围，将用于输出到外部。图像窗口设置通过：0X51/0X52/0X53/0X54/0X55/0X57 等寄存器设置。

图像输出大小设置，这是最终输出到外部的图像尺寸。该设置将图像窗口设置所决定的窗口大小，通过内部 DSP 处理，缩放成我们输出到外部的图像大小。该设置将会对图像进行缩放处理，如果设置的图像输出大小不等于图像窗口设置图像大小，那么图像就会被缩放处理，只有这两者设置一样大的时候，输出比例才是 1:1 的。

因为 OmniVision 公司公开的文档，对这些设置实在是没有详细介绍。只能从他们提供的初始化代码（还得去 linux 源码里面移植过来）里面去分析规律，所以，这几个设置，都是根据 OV2640 的调试经验，以及相关文档总结出来的，不保证百分百正确，如有错误，还请大家指正。

以上几个设置，光看文字可能不太清楚，这里我们画一个简图有助于大家理解，如下图所示：



上图，最终红色框所示的图像输出大小，才是 OV2640 输出给外部的图像尺寸，也就是显示在 LCD 上面的图像大小。当图像输出大小与图像窗口不等时，会进行缩放处理，在 LCD 上面看到的图像将会变形。

1.2.2 串行摄像头控制总线(SCCB)简介

PZ-OV2640 摄像头模块的所有配置，都是通过 SCCB 总线来实现的，SCCB 全称是：Seril Camera Control Bus 即串行摄像头控制总线，它由两条数据线组成：一个是用于传输时钟信号的 SIO_C（即 OV_SCL），另一个是用于传输数据

信号的 SIO_D (即 OV_SDA)。

SCCB 的传输协议与 IIC 协议极其相似,只不过 IIC 在每传输完一个字节后,接收数据的一方要发送一位的确认数据,而 SCCB 一次要传输 9 位数据,前 8 位为有用数据,而第 9 位数据在写周期中是 don't care 位(即不必关心位),在读周期中是 NA 位。SCCB 定义数据传输的基本单元为相(phase),即一个相传输一个字节数据。

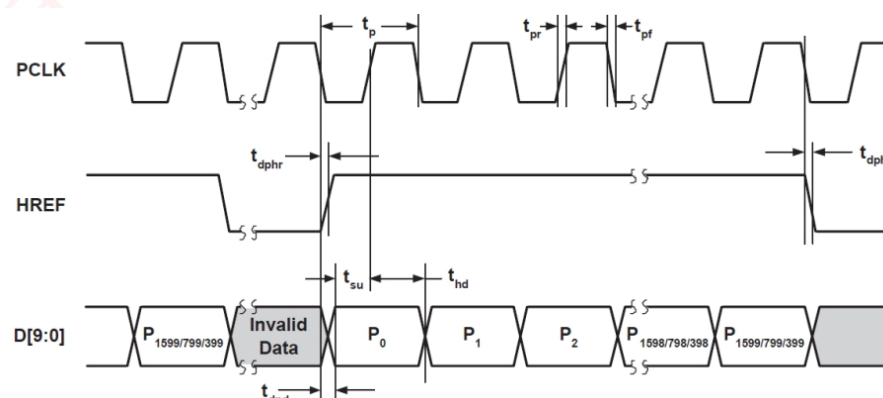
SCCB 只包括三种传输周期,即 3 相写传输周期(三个相依次为设备从地址,内存地址,所写数据),2 相写传输周期(两个相依次为设备从地址,内存地址)和 2 相读传输周期(两个相依次为设备从地址,所读数据)。当需要写操作时,应用 3 相写传输周期,当需要读操作时,依次应用 2 相写传输周期和 2 相读传输周期。

关于 SCCB 的详细介绍,请大家参考模块资料“\4--参考资料\《OmniVision Technologies Seril Camera Control Bus(SCCB) Specification.pdf》”这个文档。

OV2640 的初始化,需要配置大量寄存器,这里我们就不给大家多做介绍了,请大家参考模块资料“\4--参考资料\《OV2640 Software Application Notes 1.03.pdf》”这个文档。

1.2.3 行像素输出时序介绍

PZ-OV2640 的数据输出,是在行参考信号和像素时钟的控制下,有序输出的,默认的行像素输出时序,如下图所示:



当行参考信号 (HREF) 为高电平时,表示数据有效,此时,每输出一个像素

时钟（PCLK），就输出一个数据（8/10 位）。数据在 PCLK 的下降沿更新，所以，MCU 在 PCLK 的上升沿读取数据。

注意：图中的 t_p 表示像素周期，像素周期可能等于 1 个 PCLK，也可能等于 2 个 PCLK。在 RGB/YUV 输出格式下，每个 $t_p=2$ 个 Tpc1k，如果是 Raw RGB 格式，则一个 $t_p=1$ 个 Tpc1k，其中：Tpc1k 即 PCLK 信号的周期。

当使用 RGB565 输出的时候， $t_p=2$ 个 Tpc1k，每一个像素是 2 个字节，低字节在前，高字节在后，那么如果采用 UXGA 分辨率，那么每一行就有 1600×2 个 PCLK，用于输出行像素数据。

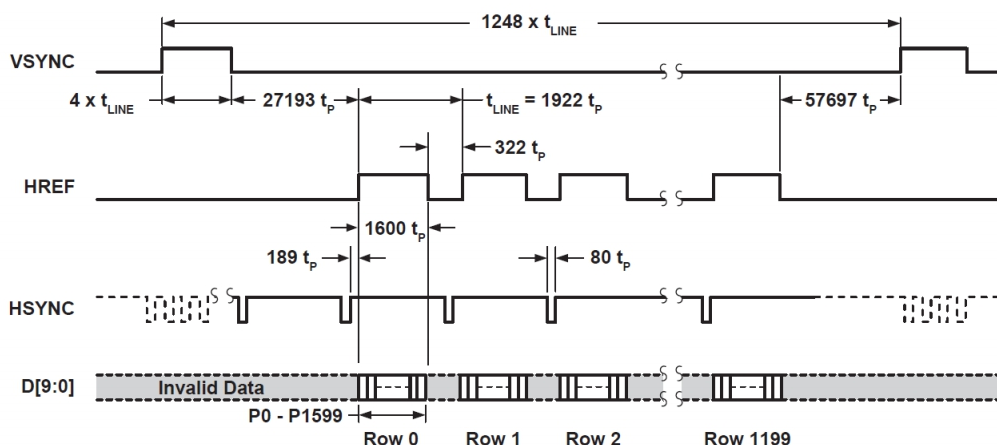
当使用 JPEG 输出的时候，输出的是压缩后的数据，这里与普通的行像素输出时序略有不同，普通的行像素输出，HREF 是连续的，也就是整个一行像素输出，HREF 会一直保持高电平，而 JPEG 输出的时候，HREF 则是不连续的，即一行数据输出时，HREF 可能会有很多次低电平出现。但是这并不影响我们读取数据，只需要判定 HREF 为高电平，再读取数据，就可以了。JPEG 输出时，不存在高低字节的概念，只需要从头到尾，将所有数据读取保存下来，就可以完成一次 JPEG 数据采集。

输出的 JPEG 数据以：0xFF, 0xD8 开头，以 0xFF, 0xD9 结尾，且在 0xFF, 0xD8 之前，或者 0xFF, 0xD9 之后，会有不定数量的其他数据存在（一般是 0），这些数据我们直接忽略即可，将得到的 0xFF, 0xD8~0xFF, 0xD9 之间的数据，保存为 .jpg/.jpeg 文件，就可以直接在电脑上打开看到图像了。

注意：PCLK 的频率可达 36Mhz，所以 MCU 在读取数据的时候，必须速度够快才可以，否则就可能出现数据丢失。对于速度不够快的 MCU，我们可以通过设置 0V2640 的寄存器（0XD3 和 0X11），设置 PCLK 和时钟的分频来降低 PCLK 速度，从而使得低速 MCU 也可以读取 0V2640 的数据。不过这样会降低帧率。

1.2.4 帧时序介绍

前面我们介绍了行像素输出时序，一帧图像数据，实际上就是由 N 个行像素输出时序组成的。这里我们以 UXGA 的帧时序为例进行介绍，UXGA 的帧时序如下图所示：



图中 t_{LINE} 即行输出时间， t_p 则是像素周期。VSYNC 是帧同步信号，每一个脉冲，表示一个新帧的开始，而整个帧周期内，有 1200 次行像素（Row）输出，每一行为 1600 个像素，这样得到的输出数据，刚好是 1600×1200 的分辨率，即 UXGA 的分辨率。

HSYNC 为行同步信号，用于同步行输出数据，不过，我们的 PZ-0V2640 模块，并没有引出该信号。大家可以用 HREF 做同步即可。

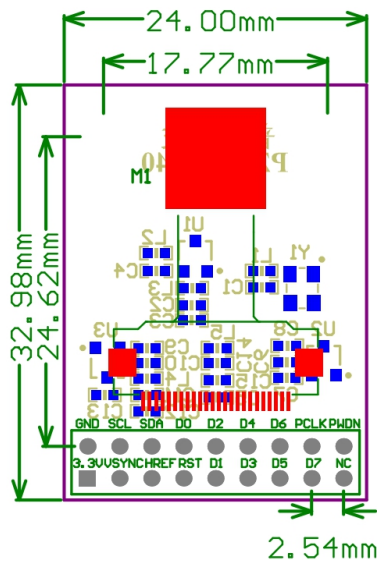
其他分辨率下，帧时序图，与 UXGA 大同小异，请大家参考模块资料“\4--参考资料\《0V2640_DS(1.6).pdf》”这个文档。

1.2.5 模块使用注意事项

PZ-0V2640 摄像头模块使用的是一体化的 0V2640 模组，本身就含有调焦镜头。当显示画面模糊时，可通过旋转模组镜头来调节清晰度。如果用手调节比较困难时，可以用镊子夹住镜头旋转。

1.3 结构尺寸

PZ-0V2640 摄像头模块尺寸结构如下图所示：



1.4 如何获取图像

下面我们来看看如何使用 PZ-0V2640 摄像头模块。对于该模块，我们只关心两点：1，如何读取 RGB565 图像数据；2，如何读取 JPEG 压缩图像数据。

先了解 3 个前提：

- 1，本例程，我们将设置 VSYNC 为高电平有效，即 VSYNC 低电平的时候，输出图像数据，高电平的时候，做帧同步信号。
- 2，本例程，我们将设置 HREF 为高电平有效，即 HREF 为高的时候，数据有效，HREF 为低的时候，数据无效。
- 3，本例程，我们将配置 PCLK 的下降沿更新数据，即 PCLK 下降沿的时候，OV2640 更新数据到 OV_D0~D7，所以，MCU 在 PCLK 的上升沿读取数据。

(1) 如何读取 RGB565 图像数据。

根据前面的 3 个前提，我们可以很容易想到读取数据的方法，因为 OV2640 的数据输出，是很有规律的，我们以行为单位，每输出一行数据，即一个 HREF 周期。OV2640 输出一行 RGB565 数据的读取过程如下：等待 VSYNC 为低电平→等待 HREF 为高电平→等待第 1 个 PCLK 的上升沿→读取第 1 个像素的低字节→等待第 2 个 PCLK 的上升沿→读取第 1 个像素的高字节→等待第 3 个 PCLK 的上升沿→读取低 2 个像素的低字节→等待第 4 个 PCLK 的上升沿……→读取 1 行数据最后 1 个像素的高字节→完成 1 行数据读取→等待 HREF 为高电平……

上面，红色部分，循环 N 次，就可以读取 N 行的数据，直到读取完整个一帧图像的输出生，最后 VSYNC 变成高电平，完成一帧图像数据的读取。当 VSYNC 再次变低的时候，开始下一帧数据输出循环。

一次数据输出多少行，每一行多少个像素，则是根据我们设定的图像输出大小（分辨率）来确定的，假定我们设定图像输出大小为：320*480，那么每一行就有 640 个 PCLK 上升沿，每一帧图像就有 480 行图像数据。

（2）如何读取 JPEG 压缩后的图像数据。

同样根据前面的 3 个前提，**不过在 JPEG 输出的时候，VSYNC 变成了：高电平时输出图像数据，低电平时数据无效，这个要特别注意下。**

OV2640 输出 JPEG 数据的读取过程如下：等待 VSYNC 为高电平→等待 HREF 为高电平→等待 PCLK 的上升沿→读取 JPEG 数据→等待 HREF 为高电平→等待 PCLK 的上升沿→读取 JPEG 数据→等待 HREF 为高电平……→VSYNC 为低电平→完成一帧 JPEG 数据读取。

从上面的流程可以看出，读取 JPEG 数据，相对读取 RGB565 数据来说，简单不少，只需要在 HREF 为高时，在 PCLK 的上升沿读取数据即可，直到 VSYNC 为低电平，完成一帧 JPEG 数据采集。

不过，需要注意的是：HREF 在一帧 JPEG 数据输出的时候，并不是持续的高电平，所以任何时候，都要先判断 HREF 为高电平，再去读取数据。

以上，就是 RGB565 输出以及 JPEG 输出时，OV2640 的图像数据读取方法。不过特别注意：OV2640 的 PCLK 速度可以很快（最高达 36Mhz），所以，代码一定要尽可能优化，而且对 PCLK 也要进行分频，才可能用在 STM32F103 这类不带 DCMI 接口的 MCU 上面。

2 硬件设计

本章实验功能简介：开机后，初始化摄像头模块（OV2640），如果初始化成功，则进入 JPEG 模式。

当使用 JPEG 模式时，图像可以设置任意尺寸（QCIF~UXGA），**注意：由于 STM32F103 内部 RAM 的限制，例程最大仅支持 VGA 尺寸(640*480)。**采集到的 JPEG

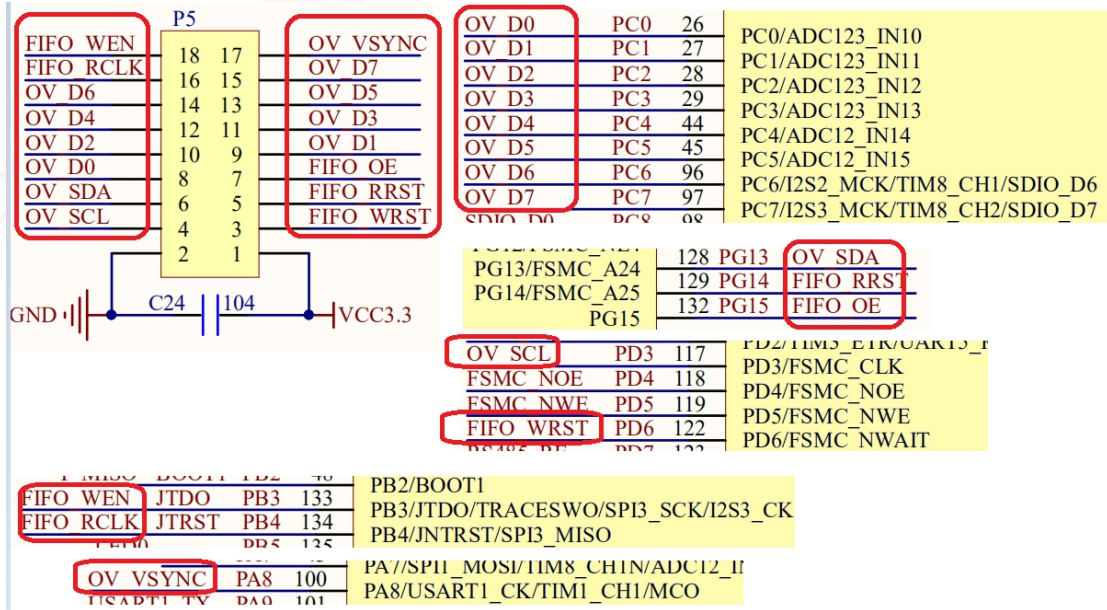
数据将先存放到 STM32 的内存里面，每当采集到一帧数据，就会将采集到的数据发送到串口 1（此时可以通过上位机软件“\3--软件工具\串口&网络摄像头软件\CAM.exe”接收，并显示图片），之后再重新采集传输。我们可以通过 KEY_UP 设置输出图片的尺寸（QCIF~VGA）。通过 KEY0 按键，可以设置对比度；KEY1 按键，可以设置饱和度；KEY2 按键，可以设置特效（注意：F103 最小系统没有 KEY2 按键）。

DS0 指示程序运行状态，DS1 用于指示帧中断。

本实验使用到硬件资源如下：

- (1) DS0、DS1 指示灯
- (2) KEY_UP、KEY0、KEY1 和 KEY2 按键
- (3) 串口 1
- (4) TFTLCD 模块
- (5) PZ-0V2640 摄像头模块

这些资源，除了最后一个，其他的我们在开发板中开发攻略教程都介绍过。这里我们重点介绍下玄武&朱雀开发板的摄像头接口与 PZ-0V2640 摄像头模块的连接。在开发板的左上角的 2*9 的 P5 排座，是摄像头模块/OLED 模块共用接口。本实验，我们只需要将 PZ-V2640 摄像头模块插入这个接口即可，该接口与 STM32 的连接关系如下图所示：



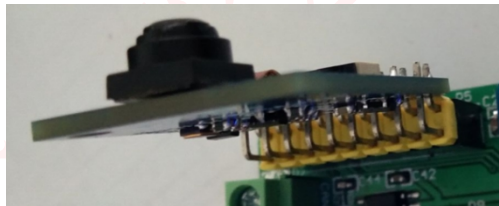
从上图可以看出，OV2640 摄像头模块的各信号脚与 STM32 的连接关系为：

OV_SDA 接 PG13;
OV_SCL 接 PD3;
FIFO_RCLK 接 PB4;
FIFO_WEN 接 PB3;
FIFO_WRST 接 PD6;
FIFO_RRST 接 PG14;
FIFO_OE 接 PG15; OV_VSYNC 接 PA8;
OV_D[7:0]接 PC[7:0];

这些线的连接，玄武&朱雀开发板的内部已经连接好了，我们只需要将 PZ-0V2640 摄像头模块插上去就好了。板载接口如下：



特别注意：模块插入开发板摄像头接口时注意方向，可对照板载接口背面 PCB 丝印电源脚位即可，PZ-0V2640 插入开发板图如下：



3 软件设计

打开本实验工程目录，我们在 APP 文件夹下新建了 OV2640 文件夹。在 ov2640 文件夹下新建了 ov2640.c、sccb.c、ov2640.h、sccb.h、ov2640cfg.h 等 5 个文件，并同时在工程中将这个文件夹加入头文件包含路径。

本实验总共新增了 5 个文件，代码比较多，我们就不一一列出了，仅挑几个重要的地方进行讲解。

3.1 ov2640.c

首先，我们来看 ov2640.c 里面的 OV2640_Init 函数，该函数代码如下：


```

1. //初始化 OV2640
2. //配置完以后,默认输出是 1600*1200 尺寸的图片!!
3. //返回值:0,成功
4. // 其他,错误代码
5. u8 OV2640_Init(void)
6. {
7.     u16 i=0;
8.     u16 reg;
9.     GPIO_InitTypeDef GPIO_InitStructure;
10. //设置 IO
11.     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB|RCC_APB2Periph_GPIOC|RCC_APB2Periph_GPIOD|R
        CC_APB2Periph_GPIOG|RCC_APB2Periph_AFIO, ENABLE); //使能相关端口时钟
12.
13.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4; //PB4 输入 上拉
14.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
15.     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
16.     GPIO_Init(GPIOB, &GPIO_InitStructure);
17.     GPIO_SetBits(GPIOB,GPIO_Pin_4);
18.
19.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3; //PB3 输出
20.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
21.     GPIO_Init(GPIOB, &GPIO_InitStructure);
22.     GPIO_SetBits(GPIOB,GPIO_Pin_3);
23.
24.     GPIO_InitStructure.GPIO_Pin = 0xff; //PC0~7 输入 上拉
25.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
26.     GPIO_Init(GPIOC, &GPIO_InitStructure);
27.
28.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
29.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU; //PD6 输入
30.     GPIO_Init(GPIOD, &GPIO_InitStructure);
31.     GPIO_SetBits(GPIOD,GPIO_Pin_6);
32.
33.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_15; //PG15 输出
34.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
35.     GPIO_Init(GPIOG, &GPIO_InitStructure);
36.     GPIO_SetBits(GPIOG,GPIO_Pin_15);
37.
38.     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_14; //PG14 输入
39.     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
40.     GPIO_Init(GPIOG, &GPIO_InitStructure);
41.     GPIO_SetBits(GPIOG,GPIO_Pin_14);
42.
43.     GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable,ENABLE);

```

```

44.
45.     OV2640_PWDN=0;           //POWER ON
46.     delay_ms(10);
47.     OV2640_RST=0;           //复位 OV2640
48.     delay_ms(10);
49.     OV2640_RST=1;           //结束复位
50.     SCCB_Init();             //初始化 SCCB 的 IO 口
51.     SCCB_WR_Reg(OV2640_DSP_RA_DLMT, 0x01); //操作 sensor 寄存器
52.     SCCB_WR_Reg(OV2640_SENSOR_COM7, 0x80); //软复位 OV2640
53.     delay_ms(50);
54.     reg=SCCB_RD_Reg(OV2640_SENSOR_MIDH); //读取厂家 ID 高八位
55.     reg<<=8;
56.     reg|=SCCB_RD_Reg(OV2640_SENSOR_MIDL); //读取厂家 ID 低八位
57.     if(reg!=OV2640_MID)
58.     {
59.         printf("MID:%d\r\n",reg);
60.         return 1;
61.     }
62.     reg=SCCB_RD_Reg(OV2640_SENSOR_PIDH); //读取厂家 ID 高八位
63.     reg<<=8;
64.     reg|=SCCB_RD_Reg(OV2640_SENSOR_PIDL); //读取厂家 ID 低八位
65.     if(reg!=OV2640_PID)
66.     {
67.         printf("HID:%d\r\n",reg);
68.         //return 2;
69.     }
70.     //初始化 OV2640,采用 SXGA 分辨率(1600*1200)
71.     for(i=0;i<sizeof(ov2640_sxga_init_reg_tbl)/2;i++)
72.     {
73.         SCCB_WR_Reg(ov2640_sxga_init_reg_tbl[i][0],ov2640_sxga_init_reg_tbl[i][1]);
74.     }
75.     return 0x00; //ok
76. }

```

此部分代码先初始化 OV2640 相关的 IO 口（包括 SCCB_Init），然后最主要的是完成 OV2640 的寄存器序列初始化。OV2640 的寄存器特多（百几十个），配置特麻烦，幸好厂家有提供参考配置序列（详见《OV2640 Software Application Notes 1.03》），本章我们用到的配置序列，存放在 ov2640_uxga_init_reg_tbl 这个数组里面，该数组是一个 2 维数组，存储初始化序列寄存器及其对应的值，该数组存放在 ov2640cfg.h 里面。

另外，在 ov2640.c 里面，还有几个函数比较重要，这里贴代码了，只介绍

功能：

OV2640_Window_Set 函数，该函数用于设置传感器输出窗口；

OV2640_ImageSize_Set 函数，用于设置图像尺寸；

OV2640_ImageWin_Set 函数，用于设置图像窗口大小；

OV2640_OutSize_Set 函数，用于设置图像输出大小；

这就是我们前面小节所介绍的 4 个设置，他们共同决定了图像的输出。

3.2 ov2640cfg.h

接下来，我们看看 ov2640cfg.h 里面 ov2640_uxga_init_reg_tbl 的内容，
ov2640cfg.h 文件的代码如下：

```
1. //OV2640 UXGA 初始化寄存器序列表
2. //此模式下帧率为 15 帧
3. //UXGA(1600*1200)
4. const u8 ov2640_uxga_init_reg_tbl[][2]=
5. {
6.     0xff, 0x00,
7.     .....//省略部分代码
8.     0x05, 0x00,
9. };
10. //OV2640 SVGA 初始化寄存器序列表
11. //此模式下, 帧率可以达到 30 帧
12. //SVGA 800*600
13. const u8 ov2640_svga_init_reg_tbl[][2]=
14. {
15.     0xff, 0x00,
16.     .....//省略部分代码
17.     0x05, 0x00,
18. };
19. const u8 ov2640_yuv422_reg_tbl[][2]=
20. {
21.     0xFF, 0x00,
22.     .....//省略部分代码
23.     0x00, 0x00,
24. };
25. const u8 ov2640_jpeg_reg_tbl[][2]=
26. {
27.     0xff, 0x01,
28.     .....//省略部分代码
29.     0xe0, 0x00,
```

```

30. };
31. const u8 ov2640_rgb565_reg_tbl[][2]=
32. {
33.     0xFF, 0x00,
34.     .....//省略部分代码
35.     0xE0, 0x00,
36. };

```

以上代码，我们省略了很多（全部贴出来太长了），里面总共有 5 个数组。我们大概了解下数组结构，每个数组条目的第一个字节为寄存器号（也就是寄存器地址），第二个字节为要设置的值，比如{0xFF, 0x01}，就表示在 0xFF 地址，写入 0x01 这个值。

五个数组里面 ov2640_uxga_init_reg_tbl 和 ov2640_svga_init_reg_tbl，分别用于配置 OV2640 输出 UXGA 和 SVGA 分辨率的图像，我们只用了 ov2640_uxga_init_reg_tbl 这个数组，完成对 OV2640 的初始化(设置为 UXGA)。最后 OV2640 要输出数据是 RGB565 还是 JPEG,就得通过其他数组设置，输出 RGB565 时，通过一个数组：ov2640_rgb565_reg_tbl 设置即可；输出 JPEG 时，则要通过 ov2640_yuv422_reg_tbl 和 ov2640_jpeg_reg_tbl 两个数组设置。

其他部分代码我们就不再细说了，请大家参考实验例程。

3.3 main.c

最后，打开 main.c 文件，代码如下：

```

1. #include "system.h"
2. #include "SysTick.h"
3. #include "led.h"
4. #include "usart.h"
5. #include "tftlcd.h"
6. #include "key.h"
7. #include "time.h"
8. #include "ov2640.h"
9.
10.
11.
12.
13. #define jpeg_buf_size 31*1024 //定义 JPEG 数据缓存 jpeg_buf 的大小
14. u8 jpeg_buf[jpeg_buf_size]; //JPEG 数据缓存 buf

```

```

15.
16.
17. //JPEG 尺寸支持列表
18. const u16 jpeg_img_size_tbl[][2]=
19. {
20.     176,144,    //QCIF
21.     160,120,    //QQVGA
22.     352,288,    //CIF
23.     320,240,    //QVGA
24.     640,480,    //VGA
25.     // 800,600,    //SVGA
26.     // 1024,768,    //XGA
27.     // 1280,1024,    //SXGA
28.     // 1600,1200,    //UXGA
29. };
30.
31. const u8*EFFECTS_TBL[7]={ "Normal", "Negative", "B&W", "Redish", "Greenish", "Bluish", "Antique"};
    //7 种特效
32. const u8*JPEG_SIZE_TBL[9]={ "QCIF", "QQVGA", "CIF", "QVGA", "VGA", "SVGA", "XGA", "SXGA", "UXGA"};
    //JPEG 图片 9 种尺寸
33.
34.
35. void jpeg_test(void)
36. {
37.     static uint32_t jpeglen=0,i=0,headok=0,jpgstart=0,jpglen=0;
38.     u8 *pbuf;
39.     u8 key=0;
40.     u8 effect=0,saturation=2,contrast=2;
41.     u8 size=3;    //默认是 QVGA 320*240 尺寸
42.     u8 msgbuf[15]; //消息缓存区
43.
44.     LCD_Clear(WHITE);
45.     FRONT_COLOR=RED;
46.     LCD_ShowString(30,50,200,16,16, "PRECHIN-STM32");
47.     LCD_ShowString(30,70,200,16,16, "OV2640 JPEG Mode");
48.     LCD_ShowString(30,100,200,16,16, "KEY0:Contrast");    //对比度
49.     LCD_ShowString(30,120,200,16,16, "KEY1:Saturation");    //色彩饱和度
50.     LCD_ShowString(30,140,200,16,16, "KEY2:Effects");    //特效
51.     LCD_ShowString(30,160,200,16,16, "KEY_UP:Size");    //分辨率设置
52.     sprintf((char*)msgbuf, "JPEG Size:%s", JPEG_SIZE_TBL[size]);
53.     LCD_ShowString(30,180,200,16,16,msgbuf);    //显示当前 JPEG 分辨率
54.
55.
56.     OV2640_JPEG_Mode();    //切换为 JPEG 模式

```

```

57.     OV2640_OutSize_Set(jpeg_img_size_tbl[size][0], jpeg_img_size_tbl[size][1]);
58.     SCCB_WR_Reg(0XFF, 0X00);
59.     SCCB_WR_Reg(0XD3, 30);
60.     SCCB_WR_Reg(0XFF, 0X01);
61.     SCCB_WR_Reg(0X11, 0X1);
62.
63.     while(1)
64.     {
65.         jpeglen=0; headok=0; jpgstart=0; jpglen=0;
66.         while(OV2640_VSYNC==1) //开始采集 jpeg 数据
67.         {
68.             while(OV2640_HREF)
69.             {
70.                 while(OV2640_PCLK==0);
71.                 jpeg_buf[jpeglen]=OV2640_DATA;
72.                 while(OV2640_PCLK==1);
73.                 jpeglen++;
74.             }
75.         }
76.
77.         pbuf=(u8*)jpeg_buf;
78.         for(i=0; i<jpeglen; i++) //查找 0XFF, 0XD8
79.         {
80.             if((pbuf[i]==0XFF)&&(pbuf[i+1]==0XD8)) //找到 FF D8
81.             {
82.                 jpgstart=i;
83.                 headok=1; //标记找到 jpg 头(FF D8)
84.             }
85.             if((pbuf[i]==0XFF)&&(pbuf[i+1]==0XD9)&&headok) //找到头以后, 再找 FF D9
86.             {
87.                 jpglen=i-jpgstart+2;
88.                 LED1=!LED1;
89.                 break;
90.             }
91.         }
92.         if(jpglen) //正常的 jpeg 数据
93.         {
94.             LCD_ShowString(30, 210, 210, 16, "Sending JPEG data..."); //提示正在传输数据
95.             pbuf+=jpgstart; //偏移到 0XFF, 0XD8 处
96.             for(i=0; i<jpglen; i++) //发送整个 jpg 文件
97.             {
98.                 while(USART_GetFlagStatus(USART1, USART_FLAG_TC)==RESET); //循环发送, 直到发
送完毕
99.                 USART_SendData(USART1, pbuf[i]);

```

```

100.         key=KEY_Scan(0);
101.         if(key)break;
102.     }
103.     if(key) //有按键按下,需要处理
104.     {
105.         LCD_ShowString(30,210,210,16,16,"Quit Sending data ");//提示退出数据传输
106.         switch(key)
107.         {
108.             case KEY0_PRESS: //对比度设置
109.                 contrast++;
110.                 if(contrast>4)contrast=0;
111.                 OV2640_Contrast(contrast);
112.                 sprintf((char*)msgbuf,"Contrast:%d",(signed char)contrast-2);
113.                 break;
114.             case KEY1_PRESS: //饱和度 Saturation
115.                 saturation++;
116.                 if(saturation>4)saturation=0;
117.                 OV2640_Color_Saturation(saturation);
118.                 sprintf((char*)msgbuf,"Saturation:%d",(signed char)saturation-2);
119.                 break;
120.             case KEY2_PRESS: //特效设置
121.                 effect++;
122.                 if(effect>6)effect=0;
123.                 OV2640_Special_Effects(effect);//设置特效
124.                 sprintf((char*)msgbuf,"%s",EFFECTS_TBL[effect]);
125.                 break;
126.             case KEY_UP_PRESS: //JPEG 输出尺寸设置
127.                 size++;
128.                 if(size>4)size=0;
129.                 OV2640_OutSize_Set(jpeg_img_size_tbl[size][0],jpeg_img_size_tbl[size][1]);//设置输出尺寸
130.                 sprintf((char*)msgbuf,"JPEG Size:%s",JPEG_SIZE_TBL[size]);
131.                 break;
132.         }
133.         LCD_Fill(30,180,239,190+16,WHITE);
134.         LCD_ShowString(30,180,210,16,16,msgbuf);//显示提示内容
135.         delay_ms(800);
136.     }else LCD_ShowString(30,210,210,16,16,"Send data complete!!");//提示传输结束设置
137.
138. }else
139.     LED2=!LED2;
140. }
141. }

```



```

142.
143. int main()
144. {
145.     u8 key;
146.     u8 t;
147.
148.     SysTick_Init(72);
149.     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2); //中断优先级分组 分2组
150.     LED_Init();
151.     USART1_Init(921600);
152.     TFTLCD_Init(); //LCD 初始化
153.     KEY_Init();
154.
155.     FRONT_COLOR=RED; //设置字体为红色
156.     LCD_ShowString(30,50,200,16,16,"PRECHIN-STM32");
157.     LCD_ShowString(30,70,200,16,16,"OV2640 TEST");
158.     LCD_ShowString(30,90,200,16,16,"www.prechin.net");
159.     while(OV2640_Init())//初始化 OV2640
160.     {
161.         LCD_ShowString(30,130,240,16,16,"OV2640 ERR");
162.         delay_ms(200);
163.         LCD_Fill(30,130,239,170,WHITE);
164.         delay_ms(200);
165.     }
166.     LCD_ShowString(30,130,200,16,16,"OV2640 OK");
167.     LCD_ShowString(30,150,230,16,16,"KEY1:JPEG");
168.
169.     while(1)
170.     {
171.         jpeg_test();
172.     }
173. }

```

这里定义了一个非常大的数组 jpeg_buf (31KB)，用来存储 JPEG 数据，由于受限于 STM32 内存容量，所以对于 VGA 以上的尺寸图像无法显示。

在 main.c 里面，总共有 2 个函数：jpeg_test 和 main 函数。其中，jpeg_test 函数将 OV2640 设置为 JPEG 模式，该函数接收 OV2640 的 JPEG 数据，并通过串口 1 发送给上位机。最后，main 函数就相对简单了，大家看代码即可。

4 实验现象

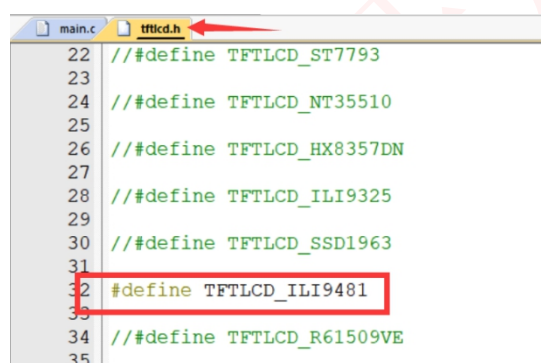
首先，请先确保硬件都已经连接好：

①PZ-OV2640 模块与玄武&朱雀开发板连接（可参考硬件设计小节）

②开发板插上 TFTLCD 液晶，对于没有 TFTLCD 的用户，可通过串口摄像头工具显示图像。

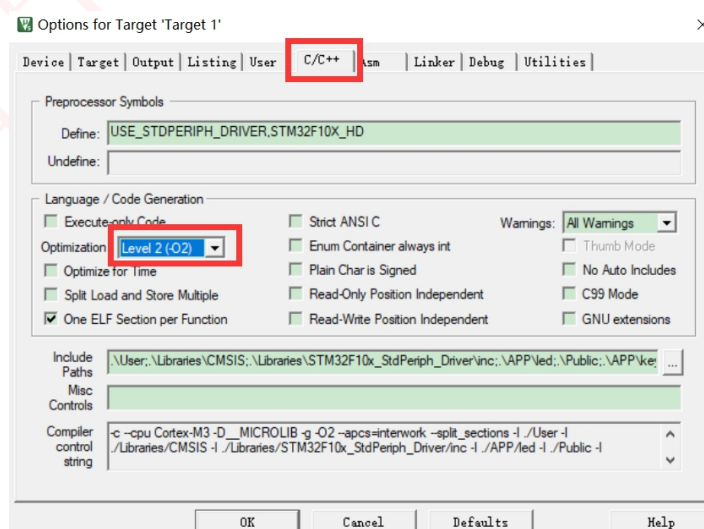
③使用 USB 线给开发板供电。

注意：如果使用普中 TFTLCD 触摸屏，用户需根据手上彩屏右上角或背面驱动型号来设置程序中使能的 TFTLCD 驱动。在 tftlcd.h 头文件中开始处进行设置，比如我使用的 TFTLCD 是 ILI9481，那么就要把这个宏定义放开，其他的注释掉。如下所示：



```
22 // #define TFTLCD_ST7793
23
24 // #define TFTLCD_NT35510
25
26 // #define TFTLCD_HX8357DN
27
28 // #define TFTLCD_ILI9325
29
30 // #define TFTLCD_SSD1963
31
32 #define TFTLCD_ILI9481
33
34 // #define TFTLCD_R61509VE
35
```

特别注意：要将工程以 -O2 等级优化编译，如果使用默认的 -O0 会显示不出图像，也不知道哪里的问题，所以移植我们工程时请注意这点。如用户知道解决方法可告知我们改进。如下：

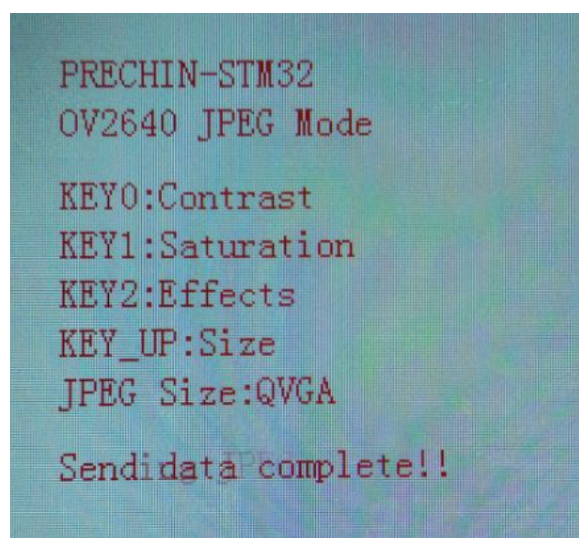


使用 -O2 优化后，要将 tftlcd.h 文件中 TFTLCD_TypeDef 结构体内程序设置

为 vu16 类型，如下：

```
main.c  tftlcd.h
35
36
37 #define TFTLCD_DIR 0 //0: 竖屏 1: 横屏 默认竖屏
38
39 #define LCD_LED PBout(0) //LCD背光 PB0
40
41 //TFTLCD地址结构体
42 typedef struct
43 {
44     vu16 LCD_CMD;
45     vu16 LCD_DATA;
46 }TFTLCD_TypeDef;
47
48
```

代码编译成功之后，我们将代码下载到 STM32 开发板上。（注意：以下的功能测试图片以玄武开发板为展示）LCD 界面显示如下图所示：



PRECHIN-STM32
OV2640 JPEG Mode

KEY0:Contrast
KEY1:Saturation
KEY2:Effects
KEY_UP:Size
JPEG Size:QVGA

Sendidata complete!!

在 OV2640 初始化成功后，进入 JPEG 模式测试。此时屏幕显示 JPEG 数据传输进程，默认条件下，图像分辨率是 QVGA(320*240)的，此时通过串口 1 发送图像数据到上位机。我们打开上位机软件“\3--软件工具\串口&网络摄像头软件\CAM.exe”，选择正确的串口，即开发板上 CH340 串口，然后波特率设置为 921600，打开即可收到 STM32 传过来的图片了，如下图所示：



我们可以通过 KEY_UP 设置输出图像的尺寸（QCIF~VGA）。通过 KEY0 按键，设置对比度；KEY1 按键，设置饱和度；KEY2 按键，设置特效。

在 time.c 的 TIM3 中断函数中，我们把帧率输出注释掉了，如果要查看多少帧，可以自行打开。

5 其他

（1）购买地址（普中授权店铺）

<http://www.prechin.net/forum.php?mod=viewthread&tid=38746&extra=>

（2）资料下载

<http://prechin.net/forum.php?mod=viewthread&tid=35264&extra=page%3D1>

（3）技术支持

普中官网：www.prechin.cn

普中论坛：www.prechin.net

技术电话：0755-21509063（转技术）